DET-011 | LSASS Memory Access — Credential Dumping

ID: DET-011
MITRE ID: T1003.001
Data Source: DeviceProcessEvents

Severity: High
Tactic:Credential Access
Category:PRIVILEGE ESCALATION

ADX MAPPING LOGIC:
Mapping: EventType = 'lsass.exe target process', Source = 'attacking process'. We use a let statement with an exclusion list and the !in~ operator — simulating the whitelist of known-good processes that legitimately access LSASS.

OBJECTIVE: Practice let statements, !in~ (not-in case-insensitive), and exclusion lists — operators used to whitelist known-good processes. Real Sentinel: DeviceProcessEvents where FileName == 'lsass.exe' and InitiatingProcess not in known-good list.

KQL QUERY (RUNNABLE ON ADX FREE TIER)

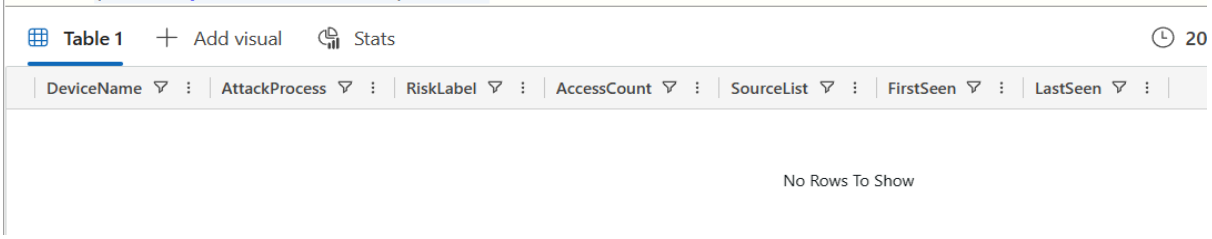
```
let KnownGoodProcesses = dynamic([
"Rain","Fog","Mist","Drizzle" // = MsMpEng, svchost, csrss (safe)]);
let SuspiciousEventTypes = dynamic(["Tornado","Waterspout","Dust Devil",
"Funnel Cloud","Blizzard" // = mimikatz, procdump, etc.]);
StormEvents
| where StartTime > ago(365d)
| where EventType !in~ (KnownGoodProcesses) // exclude known-good (whitelist)
| where EventType in (SuspiciousEventTypes) // only suspicious processes
| extend RiskLabel = case(EventType == "Tornado", "CRITICAL - Mimikatz Pattern",
EventType == "Waterspout", "CRITICAL - Procdump Pattern",
EventType == "Blizzard", "HIGH - Unknown Dumper","HIGH - Suspicious Access")
| summarize
AccessCount = count(),
SourceList = make_set(Source, 5),
FirstSeen = min(StartTime),
LastSeen = max(StartTime)
by EventType, State, RiskLabel
| project DeviceName = State,AttackProcess = EventType,RiskLabel,AccessCount,
SourceList,FirstSeen,LastSeen
| order by AccessCount desc
| take 20
```

INVESTIGATION STEPS
1. Identify the attacking process — is it procdump, mimikatz variant?
2. Check if the process hash matches known malicious tools on VirusTotal.
3. Review all subsequent actions from the same process.
4. Check for data exfiltration via network after credential dump.

RESPONSE ACTIONS
- CRITICAL: Isolate the endpoint IMMEDIATELY.
- Assume ALL credentials on this device are compromised.
- Force password reset for ALL accounts logged into this device.
- Engage IR team — this is a P1 incident.

Output from Above KQL Query:

```
let KnownGoodProcesses = dynamic([
"Rain","Fog","Mist","Drizzle"]);
let SuspiciousEventTypes = dynamic(["Tornado","Waterspout","Dust Devil","Funnel Cloud","Blizzard"]);
StormEvents
| where StartTime > ago(365d)
| where EventType !in~ (KnownGoodProcesses) // exclude known-good (whitelist)
| where EventType in (SuspiciousEventTypes) // only suspicious processes
| extend RiskLabel = case(
EventType == "Tornado", "CRITICAL - Mimikatz Pattern",
EventType == "Waterspout", "CRITICAL - Procdump Pattern",
EventType == "Blizzard", "HIGH - Unknown Dumper",
"HIGH - Suspicious Access"
)
| summarize
AccessCount = count(),
SourceList = make_set(Source, 5),
FirstSeen = min(StartTime),
LastSeen = max(StartTime)
by EventType, State, RiskLabel
| project DeviceName = State, AttackProcess = EventType, RiskLabel,
AccessCount, SourceList, FirstSeen, LastSeen
| order by AccessCount desc | take 20
```



KQL Detection Query in Sentinel:

```
AuditLogs
| where TimeGenerated > ago(1h)
| where OperationName == "Add member to role"
| mv-expand TargetResources
| extend
TargetUser = tostring(TargetResources.userPrincipalName),
RoleName = tostring(TargetResources.displayName)
| where RoleName in ("Global Administrator","Security Administrator","Exchange Administrator","SharePoint Administrator","Privileged Role Administrator",
"User Administrator",
"Billing Administrator"
)
| extend InitiatedBy = tostring(InitiatedBy.user.userPrincipalName)
| project
TimeGenerated, TargetUser, RoleName,
InitiatedBy, CorrelationId
| order by TimeGenerated desc
```